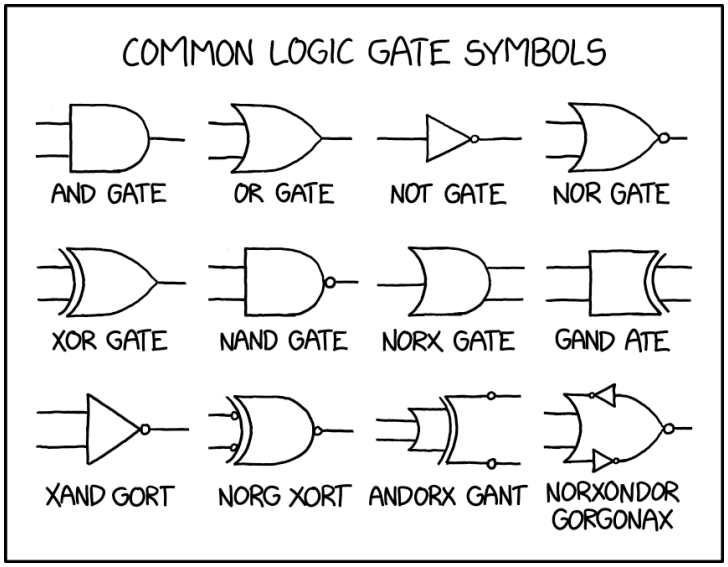
Reference and Practice Handout

Lecture-08: Boolean Logic

Computer Organization & Assembly Language Programming
CS 2160 || Fall 2026
Instructor: Himon Thakur



Logic Gates
(Randall Munroe; CC BY-NC 2.5; https://xkcd.com/2497/)

This document is a supplementary resource alongside course materials and class lectures. You are expected to go through and utilize it by yourself. You may read a section, then try solving the short exercises before moving on. Answers are provided at the end.

This handout will cover:

- What **Boolean** values are, and how hardware represents them.
- The basic operators **NOT**, **AND**, **OR** (and NAND, NOR, XOR, XNOR) as *truth tables* and *logic gates*.
- Evaluating a **Boolean function** with a truth table, using the rules of *precedence*.
- The **Boolean identities**, and using them to *simplify* expressions.
- **De Morgan's law**, in algebra and in real code.
- The two standard forms: **sum-of-products** and **product-of-sums**.

I used Claude Opus 5.5 to help me format the LaTeX code. All contents were produced, edited, or verified by me.

1. What Is Boolean Logic?

In the mid-1800s, George Boole showed that logical reasoning ("true", "false", "and", "or", "not") could be written as algebra. Almost a century later, Claude Shannon showed that electrical switching circuits follow exactly the same algebra. That idea is the foundation of every digital computer.

A **Boolean variable** has only two possible values:

Bit	Switch	Logic	Voltage
1	ON	true	high
0	OFF	false	low (about 0 V)

Older logic families used about 5 V for "high"; modern chips use lower voltages (for example 3.3 V, 1.8 V, or about 1 V inside a CPU). The exact number doesn't matter for the logic: only *high* vs. *low*.

Operators combine Boolean values, and **expressions** combine operators. A **Boolean function** has at least one variable and at least one operator; its inputs and its output are all 0 or 1. This is exactly why binary numbers suit digital hardware so well.

2. The Basic Operators and Logic Gates

A **truth table** lists the output for *every* combination of inputs. Because each input is only 0 or 1, a truth table describes an operator completely. A **logic gate** is the small circuit that computes it.

2.1. NOT, AND, OR

NOT		**AND**			**OR**		
x	$\overline{x}$	x	y	xy	x	y	$x+y$
0	1	0	0	0	0	0	0
1	0	0	1	0	0	1	**1**
		1	0	0	1	0	**1**
		1	1	**1**	1	1	**1**

- **NOT** flips the value.
- **AND** is 1 only when *both* (or all) inputs are 1. It behaves like multiplication: $1 \cdot 1 = 1$, anything with a 0 is 0.
- **OR** is 1 when *at least one* input is 1. It behaves like addition, except that $1 + 1 = 1$ (there is no "2").

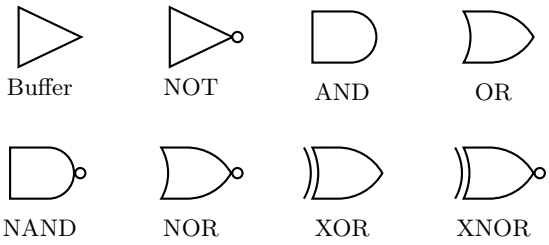
Operator	Math notation	C / Java	RISC-V (bitwise)
NOT	$\overline{x}$, $\neg x$	`~x` (bits), `!x` (logical)	`xori rd, rs, -1`
AND	xy, $x \cdot y$, $x \wedge y$	`x & y`, `x && y`	`and`, `andi`
OR	$x + y$, $x \vee y$	`x | y`, `x || y`	`or`, `ori`
XOR	$x \oplus y$	`x ^ y`	`xor`, `xori`

Connection to Lecture 04: the RISC-V `and`, `or`, `xor` instructions apply these Boolean operators to all 32 bit positions at once, one independent truth table per bit.

2.2. More gates: NAND, NOR, XOR, XNOR

x	y	**NAND** $\overline{xy}$	**NOR** $\overline{x+y}$	**XOR** $x \oplus y$	**XNOR** $\overline{x \oplus y}$
0	0	1	1	0	1
0	1	1	0	1	0
1	0	1	0	1	0
1	1	0	0	0	1

- NAND and NOR are AND and OR followed by NOT. In gate drawings, that NOT is the small **bubble** on the output.
- XOR is 1 when the inputs are *different*. XNOR is 1 when they are the *same* (an equality test).



A **buffer** outputs its input unchanged ($Y = A$). It doesn't change the logic, but real circuits use it to strengthen a signal.

Exercise 1

Fill in the outputs for following operations between $x = 1$, $y = 0$: AND ___, OR ___, NAND ___, NOR ___, XOR ___, XNOR ___.

3. Boolean Functions and Truth Tables

Precedence: NOT first, then AND, then OR, just like "negative, times, plus" in ordinary arithmetic. Use parentheses to change the order.

So $x\overline{z} + y$ means $(x \cdot \overline{z}) + y$, not $x \cdot (\overline{z} + y)$.

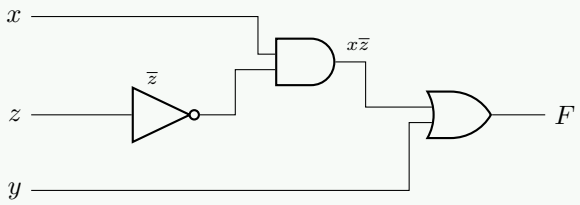
Example 1: Truth table for $F(x, y, z) = x\overline{z} + y$

With 3 inputs there are $2^3 = 8$ rows. List them in binary counting order (000, 001, ..., 111). Extra columns for the parts of the expression make the work easy to check.

x	y	z	$\overline{z}$	$x\overline{z}$	$x\overline{z} + y$
0	0	0	1	0	0
0	0	1	0	0	0
0	1	0	1	0	1
0	1	1	0	0	1
1	0	0	1	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	0	0	1

Order of work: $\overline{z}$ (NOT) $\rightarrow$ $x\overline{z}$ (AND) $\rightarrow$ $x\overline{z} + y$ (OR).

Example 2: The same function as a circuit



One NOT gate, one AND gate, one OR gate. Simpler expressions mean fewer gates: a smaller, cheaper, faster, lower-power circuit.

Exercise 2

Evaluate $H = \overline{x}y + z$ for (a) $x = 0, y = 1, z = 0$ and (b) $x = 1, y = 1, z = 0$.

Exercise 3

Make the full 8-row truth table for $G(x, y, z) = (x + y)\overline{z}$.

4. Boolean Identities

Identities are rewrite rules: the left and right sides give the same output for *every* input. Most come in pairs: an **AND form** and an **OR form**. You get one from the other by swapping AND $\leftrightarrow$ OR and $0 \leftrightarrow 1$.

Name	AND form	OR form	In words
Identity	$1x = x$	$0 + x = x$	1 for AND and 0 for OR change nothing
Null (Domination)	$0x = 0$	$1 + x = 1$	0 for AND and 1 for OR decide the result
Idempotent	$xx = x$	$x + x = x$	repeating a variable changes nothing
Inverse	$x\overline{x} = 0$	$x + \overline{x} = 1$	a value and its opposite
Commutative	$xy = yx$	$x + y = y + x$	order doesn't matter
Associative	$(xy)z = x(yz)$	$(x + y) + z = x + (y + z)$	grouping doesn't matter
Distributive	$x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$	"multiply out" (both ways!)
Absorption	$x(x + y) = x$	$x + xy = x$	the extra term is swallowed
De Morgan	$\overline{xy} = \overline{x} + \overline{y}$	$\overline{x + y} = \overline{x}\,\overline{y}$	NOT flips AND $\leftrightarrow$ OR
Double complement	$\overline{\overline{x}} = x$		two NOTs cancel

Watch out

The OR form of the distributive law, $x+yz = (x+y)(x+z)$, has **no equivalent in ordinary arithmetic** ($2+3\cdot4 \neq 5\cdot6$). It is true in Boolean algebra because $1 + 1 = 1$. Don't let your arithmetic instincts stop you from using it.

Example 3: Proving an identity with a truth table ("perfect induction")

Check $x + yz = (x + y)(x + z)$ on all 8 inputs:

x	y	z	yz	$x + yz$	$x + y$	$x + z$	$(x + y)(x + z)$
0	0	0	0	**0**	0	0	**0**
0	0	1	0	**0**	0	1	**0**
0	1	0	0	**0**	1	0	**0**
0	1	1	1	**1**	1	1	**1**
1	0	0	0	**1**	1	1	**1**
1	0	1	0	**1**	1	1	**1**
1	1	0	0	**1**	1	1	**1**
1	1	1	1	**1**	1	1	**1**

The two bold columns match in every row, so the identity holds.

Exercise 4

Prove the absorption law $x + xy = x$ using only the identity, distributive, and null laws. (Hint: start by writing x as $x \cdot 1$.)

5. Simplifying Expressions

Simplifying means rewriting an expression with fewer operators while keeping the same truth table. Apply one identity per step, and name it, so every step can be checked.

Example 4: $x + \overline{x}\,\overline{y}$

$$
\begin{aligned}
& x + \overline{x}\,\overline{y} \\
&= x + (\overline{x} + \overline{y}) && \text{(De Morgan)} \\
&= (x + \overline{x}) + \overline{y} && \text{(Associative)} \\
&= 1 + \overline{y} && \text{(Inverse)} \\
&= 1 && \text{(Null)}
\end{aligned}
$$

The whole expression is always 1, no matter what x and y are.

Example 5: $\overline{x}\,\overline{y}(\overline{x} + y)(\overline{y} + y)$

$$
\begin{aligned}
& \overline{x}\,\overline{y}(\overline{x} + y)(\overline{y} + y) \\
&= \overline{x}\,\overline{y}(\overline{x} + y) \cdot 1 && \text{(Inverse)} \\
&= \overline{x}\,\overline{y}(\overline{x} + y) && \text{(Identity)} \\
&= (\overline{x} + \overline{y})(\overline{x} + y) && \text{(De Morgan)} \\
&= \overline{x}\,\overline{x} + \overline{x}y + \overline{y}\,\overline{x} + \overline{y}y && \text{(Distributive)} \\
&= \overline{x} + \overline{x}y + \overline{x}\,\overline{y} + \overline{y}y && \text{(Idempotent, Commutative)} \\
&= \overline{x} + \overline{y}y && \text{(Absorption, twice)} \\
&= \overline{x} + 0 && \text{(Inverse)} \\
&= \overline{x} && \text{(Identity)}
\end{aligned}
$$

Example 6: $F(x,y,z) = xy + \overline{x}z + yz$ **(the "consensus" term yz is redundant)**

$$
\begin{aligned}
& xy + \overline{x}z + yz \\
&= xy + \overline{x}z + yz \cdot 1 && \text{(Identity)} \\
&= xy + \overline{x}z + yz(x + \overline{x}) && \text{(Inverse)} \\
&= xy + \overline{x}z + xyz + \overline{x}yz && \text{(Distributive, Commutative)} \\
&= (xy + xyz) + (\overline{x}z + \overline{x}zy) && \text{(Commutative, Associative)} \\
&= xy(1 + z) + \overline{x}z(1 + y) && \text{(Distributive)} \\
&= xy \cdot 1 + \overline{x}z \cdot 1 && \text{(Null)} \\
&= xy + \overline{x}z && \text{(Identity)}
\end{aligned}
$$

The trick in line 3: multiply by a clever form of 1 $(x + \overline{x})$ to split yz into two pieces that each get absorbed.

Exercise 5

Simplify (a) $xy + x\overline{y}$ and (b) $(x + y)(x + \overline{y})$. Name each law you use.

6. De Morgan's Law

> **To push a NOT inside parentheses:** complement every variable, **and** swap every AND $\leftrightarrow$ OR.
>
> $$\overline{xy} = \overline{x} + \overline{y} \qquad \overline{x + y} = \overline{x}\,\overline{y}$$

In words: "not (both)" means "at least one is not"; "not (either)" means "neither".

Example 7: Complement of $F(x,y,z) = xy + \overline{x}y + x\overline{z}$

$$
\begin{aligned}
\overline{F} &= \overline{xy + \overline{x}y + x\overline{z}} \\
&= \overline{xy}\,\overline{\overline{x}y}\,\overline{x\overline{z}} && \text{(De Morgan on the ORs)} \\
&= (\overline{x} + \overline{y})(x + \overline{y})(\overline{x} + z) && \text{(De Morgan on each AND, double complement)}
\end{aligned}
$$

Sometimes it is cheaper to build $\overline{F}$ and add one NOT gate at the end than to build F directly. De Morgan makes finding $\overline{F}$ mechanical.

Example 8: De Morgan in real code

Checking that an index is *out* of bounds:

```
if (!(i >= 0 && i < n)) { ... }      // "not inside the range"
if (i < 0 || i >= n) { ... }         // same test, after De Morgan
```

The `!` moves inside, `&&` becomes `||`, and each comparison flips ($\geq \to <$, $< \to \geq$).
In RISC-V, a bitwise NAND of `a0` and `a1` takes two instructions:

```
and   a2, a0, a1      # a2 = a0 & a1
xori  a2, a2, -1      # a2 = ~(a0 & a1)    (same as ~a0 | ~a1)
```

Exercise 6

Use De Morgan to find the complement of $x + \overline{y}z$.

Exercise 7

Rewrite the C condition `!(a && !b)` without the outer `!`.

Exercise 8

Write two RISC-V instructions that put the bitwise NOR of `a0` and `a1` into `a2`.

7. Standard Forms: Sum-of-Products and Product-of-Sums

Many different expressions can have the same truth table. A **canonical form** is one standard way of writing it that you can read straight off the truth table.

- **Sum-of-products (SOP):** an OR of AND-terms, e.g., $xy + xz + yz$. Handy when the function is 1 for only a few rows.
- **Product-of-sums (POS):** an AND of OR-terms, e.g., $(x + y)(x + z)(y + z)$. Handy when the function is 0 for only a few rows.

7.1. Recipe: truth table → SOP (use the 1-rows)

1. Find every row where $F = 1$.
2. For each such row, write an AND-term (a **minterm**): a variable that is 1 appears plain; a variable that is 0 appears complemented. (This term is 1 for that row only.)
3. OR all the terms together.

7.2. Recipe: truth table → POS (use the 0-rows)

1. Find every row where $F = 0$.
2. For each such row, write an OR-term (a **maxterm**): a variable that is 0 appears plain; a variable that is 1 appears complemented. (This term is 0 for that row only.)
3. AND all the terms together.

Example 9: Both forms for $F(x, y, z) = x\overline{z} + y$

Row	x	y	z	F	Minterm (if $F = 1$)	Maxterm (if $F = 0$)
0	0	0	0	0		$x + y + z$
1	0	0	1	0		$x + y + \overline{z}$
2	0	1	0	1	$\overline{x}y\overline{z}$	
3	0	1	1	1	$\overline{x}yz$	
4	1	0	0	1	$x\overline{y}\,\overline{z}$	
5	1	0	1	0		$\overline{x} + y + \overline{z}$
6	1	1	0	1	$xy\overline{z}$	
7	1	1	1	1	xyz	

SOP: $F = \overline{x}y\overline{z} + \overline{x}yz + x\overline{y}\,\overline{z} + xy\overline{z} + xyz$

POS: $F = (x + y + z)(x + y + \overline{z})(\overline{x} + y + \overline{z})$

Neither is the *simplest* form (that is $x\overline{z} + y$); they are *standard* forms. Shorthand you may see: $F = \Sigma m(2, 3, 4, 6, 7) = \Pi M(0, 1, 5)$, listing the row numbers.

Exercise 9

Write XOR $(x \oplus y)$ in SOP form and in POS form.

Exercise 10

The *majority* function $M(x, y, z)$ is 1 when at least two inputs are 1. Write its SOP form, then simplify it.

Recap

Remember these

1. A Boolean value is 0 or 1. In hardware: low or high voltage.
2. NOT flips; AND is 1 only if all inputs are 1; OR is 1 if any input is 1; XOR is 1 if the inputs differ.
3. A bubble on a gate's output means NOT: NAND $= \overline{xy}$, NOR $= \overline{x + y}$.
4. Precedence: NOT, then AND, then OR.
5. n inputs $\to 2^n$ rows in the truth table. Two expressions are equal if their truth tables match on every row.
6. Identities come in AND/OR pairs. Watch for $x + yz = (x + y)(x + z)$, which ordinary algebra doesn't have.
7. De Morgan: complement each variable and swap AND $\leftrightarrow$ OR. Also works on `&&`/`||` in code.
8. SOP: OR together a minterm for each 1-row. POS: AND together a maxterm for each 0-row.
9. Simpler expression $\to$ fewer gates $\to$ cheaper, faster, lower-power circuit.

Answer key for Exercises

Ex. 1. $x = 1, y = 0$: AND 0, OR 1, NAND 1, NOR 0, XOR 1, XNOR 0.

Ex. 2. (a) $\overline{x} = 1$, so $\overline{x}y = 1 \cdot 1 = 1$, and $H = 1 + 0 = \mathbf{1}$. (b) $\overline{x} = 0$, so $\overline{x}y = 0$, and $H = 0 + 0 = \mathbf{0}$.

Ex. 3. $G = (x + y)\overline{z}$ is 1 only when $z = 0$ and at least one of x, y is 1.
Rows $xyz = 000, 001, \ldots, 111$: $G = 0, 0, 1, 0, 1, 0, 1, 0$.

Ex. 4. $x + xy = x \cdot 1 + xy$ (Identity) $= x(1 + y)$ (Distributive) $= x \cdot 1$ (Null) $= x$ (Identity).

Ex. 5. (a) $xy + x\overline{y} = x(y + \overline{y})$ (Distributive) $= x \cdot 1$ (Inverse) $= x$ (Identity).
(b) $(x + y)(x + \overline{y}) = x + y\overline{y}$ (Distributive, OR form) $= x + 0$ (Inverse) $= x$ (Identity).

Ex. 6. $\overline{x + \overline{y}z} = \overline{x}\,\overline{\overline{y}z}$ (De Morgan) $= \overline{x}(y + \overline{z})$ (De Morgan, double complement).

Ex. 7. `!(a && !b)` = `!a || !!b` = `!a || b`.

Ex. 8. `or a2, a0, a1` then `xori a2, a2, -1`.

Ex. 9. XOR is 1 on rows 01 and 10, and 0 on rows 00 and 11.
SOP: $x \oplus y = \overline{x}y + x\overline{y}$. POS: $x \oplus y = (x + y)(\overline{x} + \overline{y})$.

Ex. 10. $M = 1$ on rows 3, 5, 6, 7 (011, 101, 110, 111).
SOP: $M = \overline{x}yz + x\overline{y}z + xy\overline{z} + xyz$.
Simplify: copy xyz twice (Idempotent: $xyz = xyz + xyz + xyz$) and pair it with each other term:
$(\overline{x}yz + xyz) + (x\overline{y}z + xyz) + (xy\overline{z} + xyz) = yz(\overline{x} + x) + xz(\overline{y} + y) + xy(\overline{z} + z) = \mathbf{yz + xz + xy}$.

Student Practice Worksheet

Part 1: Truth Tables

Q1. Complete the truth table for $F(x, y, z) = \overline{x}z + xy$.

x	y	z	$\overline{x}z$	xy	F
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

Q2. Show that NAND alone can build every other gate. Using only NAND, write expressions for (a) NOT x, (b) x AND y, (c) x OR y.

Part 2: Algebra

Q3. Name the law used in each step:

$$
\begin{aligned}
& x(\overline{x} + y) \\
&= x\overline{x} + xy && \text{_____} \\
&= 0 + xy && \text{_____} \\
&= xy && \text{_____}
\end{aligned}
$$

Q4. Simplify $\overline{x}\,\overline{y}z + \overline{x}yz + x\overline{y}z$ to a sum of two terms.

Q5. Use De Morgan to find $\overline{F}$ for $F = (x + y)(\overline{z} + w)$.

Part 3: Standard Forms & Circuits

Q6. The function $P(x, y, z)$ is 1 exactly when an *odd* number of inputs are 1 (rows 1, 2, 4, 7). Write P in (a) SOP form and (b) POS form.

Q7. Count the 2-input gates (and NOT gates) needed to build $xy + \overline{x}z + yz$ directly, and then after simplifying it to $xy + \overline{x}z$. How many gates did simplifying save?

Q8. Rewrite this C condition without the `!`, using De Morgan:

```
if (!(x >= 0 && x < n)) { error(); }
```

Step-by-Step Solutions & Explanations

Q1: Truth table for $\overline{x}z + xy$

x	y	z	$\overline{x}z$	xy	F
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	1	1	0	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	0	1	1
1	1	1	0	1	1

When $x = 0$, F copies z; when $x = 1$, F copies y. (This is a 2-to-1 *multiplexer*: x selects which input passes through.)

Q2: NAND is universal (write $\text{NAND}(a, b) = \overline{ab}$)

- (a) NOT: $\text{NAND}(x, x) = \overline{xx} = \overline{x}$ (Idempotent).
- (b) AND: $\text{NAND}\big(\text{NAND}(x, y), \text{NAND}(x, y)\big) = \overline{\overline{xy}} = xy$ (NOT of a NAND).
- (c) OR: $\text{NAND}(\overline{x}, \overline{y}) = \overline{\overline{x}\,\overline{y}} = x + y$ (De Morgan), with $\overline{x}$ and $\overline{y}$ made as in (a).

Q3: Name the laws

- $x(\overline{x} + y) = x\overline{x} + xy$: **Distributive**.
- $x\overline{x} + xy = 0 + xy$: **Inverse**.
- $0 + xy = xy$: **Identity**.

Q4: Simplify $\overline{x}\,\overline{y}z + \overline{x}yz + x\overline{y}z$

- Copy $\overline{x}\,\overline{y}z$ (Idempotent): $(\overline{x}\,\overline{y}z + \overline{x}yz) + (\overline{x}\,\overline{y}z + x\overline{y}z)$.
- Factor each pair (Distributive): $\overline{x}z(\overline{y} + y) + \overline{y}z(\overline{x} + x)$.
- Inverse, Identity: $\mathbf{\overline{x}z + \overline{y}z}$ (which also equals $z(\overline{x} + \overline{y})$).

Q5: Complement of $(x + y)(\overline{z} + w)$

- $\overline{F} = \overline{x + y} + \overline{\overline{z} + w}$ (De Morgan on the AND).
- $= \overline{x}\,\overline{y} + z\overline{w}$ (De Morgan on each OR, double complement).

Q6: Odd-parity function

- 1-rows: 001, 010, 100, 111. (a) SOP: $P = \overline{x}\,\overline{y}z + \overline{x}y\overline{z} + x\overline{y}\,\overline{z} + xyz$.
- 0-rows: 000, 011, 101, 110. (b) POS: $P = (x + y + z)(x + \overline{y} + \overline{z})(\overline{x} + y + \overline{z})(\overline{x} + \overline{y} + z)$.
- (This is $P = x \oplus y \oplus z$; it is used for parity-check bits in memory and networking.)

Q7: Gate count

- $xy + \overline{x}z + yz$: 1 NOT ($\overline{x}$), 3 AND, 2 OR $\rightarrow$ **6 gates**.
- $xy + \overline{x}z$: 1 NOT, 2 AND, 1 OR $\rightarrow$ **4 gates**.
- Simplifying saved **2 gates** (one AND, one OR), with exactly the same truth table.

Q8: De Morgan in C

```
if (x < 0 || x >= n) { error(); }
```

The `!` distributes: `&&` $\rightarrow$ `||`, `x >= 0` $\rightarrow$ `x < 0`, and `x < n` $\rightarrow$ `x >= n`.